

Queue Use

In order to use the queue, we need to decide whether we want our code to block and wait for data to be available (for dequeuing) or space to be available (for enqueueing). If our code should not block, then we will simply test the condition to determine whether to perform the queue operation. If the condition is true, then the code continues with the queue operation. If the condition is not true, then the code needs to defer this work for later. The actual approach will depend on the specifics of the application.

If our code should block, then we use a loop to wait for the appropriate condition to become true before performing the queue operation. Note that this blocking operation does not share the processor. When using a finite state machine to allow other code to run, this test code should be a separate state. If the condition is not true yet, the state should end and be repeated on the next call. If the condition is true, the code can continue with the queue operation and then advance to the next state. When using a task scheduler, if the condition is not true, the code should yield the processor briefly to other tasks.

Operations that might block should not be performed in interrupt handlers because they can introduce sporadic timing delays that are difficult to repeat and therefore debug. Instead, the handler should be prepared to discard the data (or handle it in some other way) and signal an error to the rest of the application.

Serial Communication Protocols and Peripherals

We can now examine three types of serial communication protocols and the corresponding peripherals. We start with a basic approach (synchronous serial), then advance to asynchronous serial and finish with I²C, which offers addressing and other higher-level features. Figure 8.8 shows the connections on the Nucleo-64 board for the SPI, USART and I²C examples in this chapter.

Synchronous Serial Communication

Protocol Concepts

Serial peripheral interface (SPI) is a type of synchronous serial communication with a master and one or more slaves (Figure 8.9). Typically the MCU is the master and peripheral devices are slaves. Some common slave SPI devices are ADCs, accelerometers, LCD controllers, and magnetometers. An MCU might instead be configured to operate as a slave, for example, to create a smart MCU-based peripheral subsystem in a larger system with a different MCU serving as the master.

SPI communication between a master and slaves uses three signals (clock and two data signals), a select signal for each slave and ground.

- The clock signal (SPSCK or SCK) indicates when data is to be sampled.
- The MOSI data signal is the master output and slave input.
- The MISO data signal is the master input and slave output.
- The master asserts the select line of the slave targeted for communication. This signal is typically active-low.

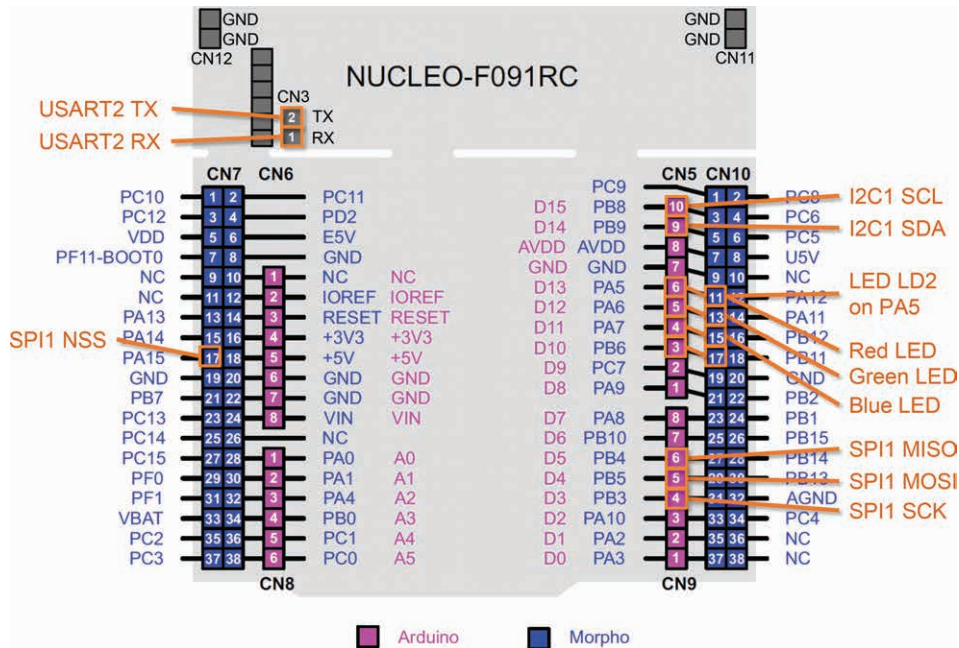


Figure 8.8 Connections on Nucleo-64 board used for SPI, USART and I²C communication examples in this chapter. Note that other port pins also have communication capabilities which are not shown here.

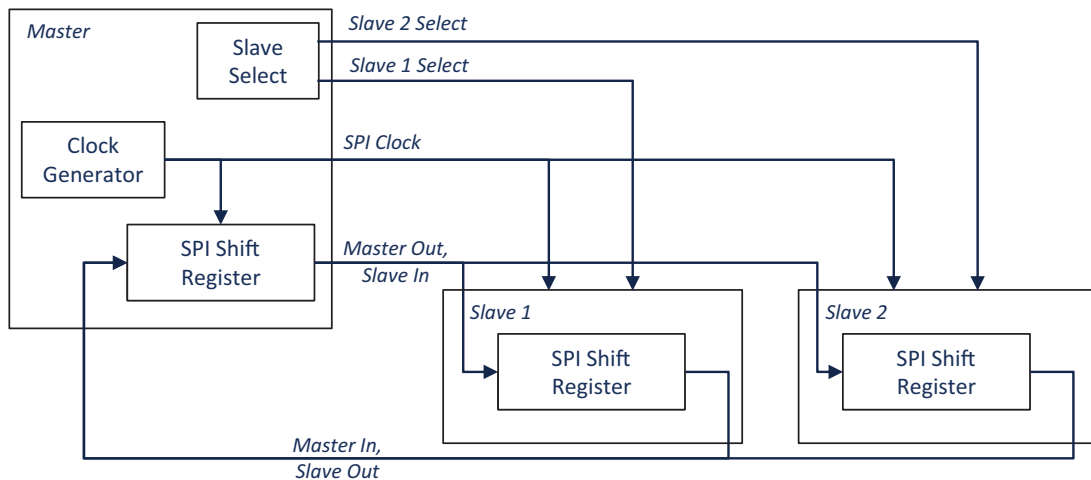


Figure 8.9 Overview of SPI system with master and two slave devices.

If bidirectional communication between the master and slaves is not needed, only one data line is needed (MOSI or MISO), depending on data transfer direction.

Communication

Figure 8.10 shows an example of communication between a master and a slave. The master selects a particular slave by asserting its slave-select line. The master asserts the clock signal to indicate when its data output signal (MOSI) is valid and should be sampled by the slave. At the same time, the master can receive data from the slave on the MISO signal (not shown in the figure). Each clock pulse exchanges one bit between the shift registers of the master and the slave. For byte SPI transmissions, one byte is exchanged with every eight clock pulses. After the last new bit is shifted in, the receiver sets a status flag indicating completed reception. It may also generate an interrupt request. Finally, the slave select line can be released, though in some cases it is not, as described in further sections.

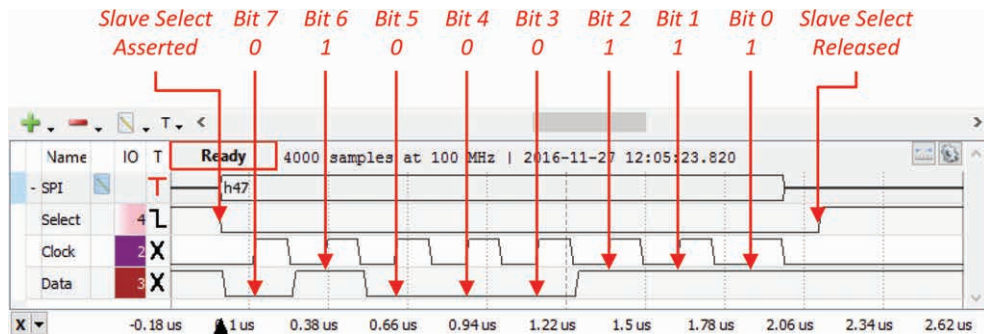


Figure 8.10 Master sending data 0x47 (01000111) to slave. Data is valid on the rising edge of the clock signal.

Clock Phase and Polarity

There are four different versions of SPI, based on the relationship between the clock and the data. Different peripherals may use different versions of SPI, so it is important to select the correct version. The clock polarity determines whether the clock signal is active-high or active-low.

The clock phase determines when the slave starts transmitting valid data on the MISO signal. In one case, the slave and master both transmit valid data when slave select is asserted. The first clock edge from inactive to active indicates the middle of the bit time, causing the master to sample MISO and the slave to sample MOSI. The next clock edge (from active to inactive) advances the shift registers. The slave select signal can remain active between transfers.

In the other case, data is not transmitted on MOSI or MISO until the inactive-to-active clock edge. It is sampled on the active-to-inactive clock edge. In addition, the slave select signal needs to go inactive between transfers.

STM32F091RC SPI Peripherals

The STM32F091RC microcontroller has two identical SPI peripherals, called SPI1 and SPI2. Each has the structure shown in Figure 8.11, with the SPI shift register at the core. When the peripheral operates in the master mode, the shift register is clocked by the clock generator module, while in slave mode the master provides the clock signal. The SPIx_DR register is used for both

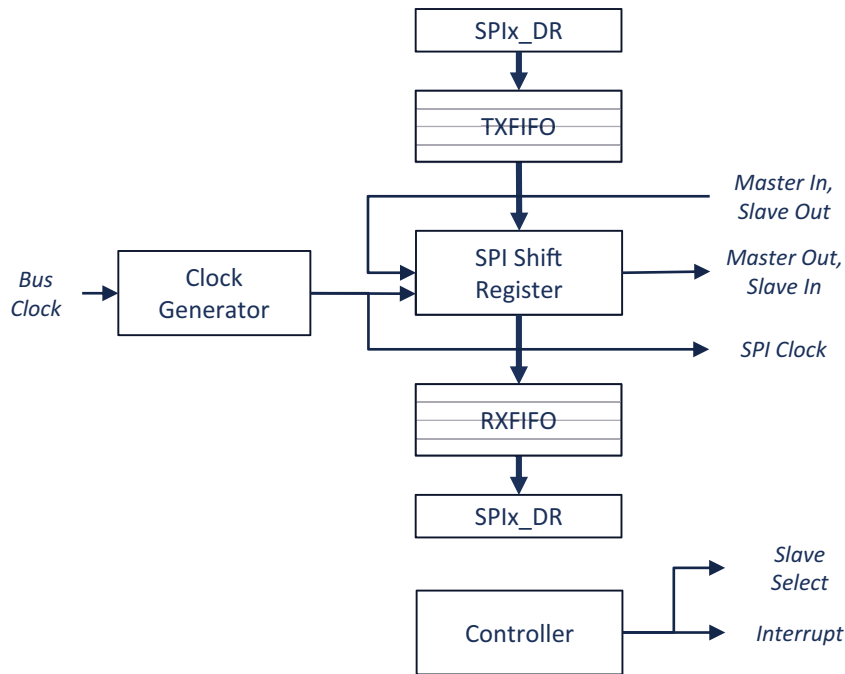


Figure 8.11 The STM32F091RC SPI peripheral structure is built around the SPI shift register.

transmission (by writing) and reception (by reading). There are two FIFO (first-in, first-out) buffers called TXFIFO and RXFIFO. These can hold up to four bytes of data each, easing the timing requirements on the software handling the SPI activities.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	BIDI MODE	BIDI OE	CRC EN	CRC NEXT	CRCL	RX ONLY	SSM	SSI	LSB FIRST	SPE	BR			MSTR	CPOL	CPHA
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8.12 SPIx_CR1 control register configures the peripheral.

The SPIx_CR1 control register, shown in Figure 8.12, configures various aspects of the peripheral. Some of the most important fields are these:

- MSTR selects whether the SPI module acts as a master (one) or slave (zero).
- LSBFIRST controls which data bit is transmitted first: the LSB (one) or the MSB (zero).
- CPOL and CPHA define clock polarity and phase. A CPOL value of zero selects an active-high clock, whereas one selects an active-low clock. A CPHA value of zero indicates valid data is transmitted starting with the slave select signal, so data can be sampled on the first clock edge (inactive to active). A value of one causes data to be sampled on the second clock edge (active to inactive).

- BR controls the baud rate if the SPI peripheral is in master mode. The master sets the rate at which data bits are shifted across the MOSI and MISO signals (Figure 8.12). The master's baud rate is derived by dividing the bus clock by a factor of 2^{1-2^8} (i.e. 2, 4, 8, 16, ..., 256). The division factor is 2^{BR+1} . The resulting baud rate is:

$$\text{Baud Rate} = \frac{f_{\text{BusClock}}}{2^{BR+1}}$$

- When one, SPE enables the SPI peripheral to operate. This also disables certain peripheral configuration changes, so SPE must be set to one only after the SPI peripheral is configured.
- Other fields control features such as bidirectional communication and CRC configuration. They are described in Chapter 28 of the reference manual.

The SPIx_CR2 register controls further peripheral configuration, shown in Figure 8.13.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	LDMA_TX	LDMA_RX	FRXTH	DS				TXEIE	RXNEIE	ERRIE	FRF	NSSP	SSOE	TXDMAEN	RXDMAEN
Reset		0	0	0	0	1	1	1	0	0	0	0	0	0	0	0
Access		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8.13 SPIx_CR2 register controls further configurations of the peripheral.

- The DS field sets the length of the data to transmit and receive. After reset, the default data length is 8 bits. Data lengths from four to sixteen bits are available.
- Three fields (TXEIE, RXNEIE and ERRIE) can enable interrupts based on transmit, receive or error activity. These are discussed further below.
- Several fields are used for DMA, which is discussed in Chapter 9. These include TXDMAEN, RXDMAEN, LDMA_TX and LDMA_RX.
- Other fields control the slave select signal and are discussed in detail in the manual.

Status and Interrupts

The SPI peripheral indicates events using status flags and interrupts. The SPIx_SR register, shown in Figure 8.14, holds status flags.

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	FTLV		FRLVL		FRE	BSY	OVR	MODF	CRCERR	UDR	CHSIDE	TXE	RXNE
Reset				0	0	0	0	0	0	0	0	0	0	0	1	0
Access				r	r	r	r	r	r	r	r	rc_w0	r	r	r	r

Figure 8.14 SPIx_SR status register indicates which SPI events have occurred.

- The TXE bit will be set when the SPIx transmission buffer is empty.
- The FTLVL and FRLVL bits indicates the level (capacity used) for the transmission FIFO and reception FIFO respectively. 00 indicates the FIFO is empty, 01 indicates the FIFO is ¼ full, 10 indicates FIFO is ½ full and 11 indicates that the FIFO is more than ½ full.
- The RXNE bit will be set when the SPIx receive FIFO is not empty; that is, it contains at least one data item. Note that data longer than 8 bits requires two bytes in the FIFO, so be sure to configure the FRXTH (FIFO RX Threshold) field in SPIx_CR2 to zero for data longer than eight bits and one otherwise.
- The BSY bit indicates SPI peripheral is busy with communication or the transmission buffer is not empty.
- The MODF bit indicates a mode fault, in which multiple masters attempt to drive the SPI clock and MOSI signals.
- Other flags indicate errors such as a CRC error (if used) or frame format error.

The SPI module can generate an interrupt request under certain conditions. Two important conditions are:

- If the transmission buffer empty interrupt enable (TXEIE of SPIx_CR2 register) is one, then an interrupt will generated when TXE of SPIx_SR register is set.
- If RXNEIE of SPIx_CR2 register is one, then an interrupt will be generated when RXNE of SPIx_SR register is set.

Transmission/Reception Activity

```
uint8_t SPI_Send_Receive_Byte(uint8_t d_out) {
    uint8_t d_in;
    // Wait until transmitter buffer is empty
    while ((SPI1->SR & SPI_SR_TXE) == 0)
        ;
    // Transmit d_out
    // Must tell compiler to use a byte write (not half-word)
    // by casting SPI1->DR into a pointer to a byte (uint8_t).
    // See STM32F0 Snippets (SPI_01_FullDuplexCommunications).
    *((uint8_t *)&(SPI1->DR)) = d_out;
    // Wait until receiver is not empty
    while ((SPI1->SR & SPI_SR_RXNE) == 0)
        ;
    // Get d_in
    d_in = (uint8_t) SPI1->DR;
    return d_in;
};
```

Listing 8.4 Code to transmit and receive one byte with SPI using polling.

Listing 8.4 shows the basic software to transmit and receive SPI data using polling. Before the MCU can transmit, it must first wait until the transmitter buffer is empty, which is indicated by the TXE flag. The MCU can then write the byte to the SPI data register. This write triggers the simultaneous transmission and reception of bytes to MOSI and from MISO and also clears the TXE flag.

When the entire byte has been received, the receiver buffer is marked as not being empty, indicated by the RXNE flag. The MCU can then read the received data from the SPI data register, which also clears the RXNE flag.

Note that the write to the SPI data register is not simply `SPI1->DR = d_out`. Instead, the code `*((uint8_t *)&(SPI1->DR)) = d_out` is used, telling the compiler to treat `SPI1->DR` as an eight-bit unsigned integer, rather than the default of 32 bits as defined in `stm32f091xc.h`. This is called “casting” the data type (in this case from `uint32_t` to `uint8_t`).

This is needed because the SPI hardware supports packing mode: when the data frame size is 8 bits or less, using a 16-bit access to `SPIx_DR` tells the hardware that two data frames have been packed into the 16 bits. The compiler will translate `SPI1->DR = d_out` to use a 16-bit store halfword instruction (STRH). If you examine MOSI with the logic analyzer, you’ll see that an extra empty byte is transmitted after the correct byte. Using the casting code tells the compiler to use a store byte instruction (STRB) instead of store halfword (STRH) or store word (STR).

In many cases interrupts can improve system responsiveness.³ Recall that data is loaded into the transmit buffer by writing to `SPIx_DR`, which in turn starts simultaneous data transmission and reception. When the byte exchange is complete, the receive buffer is full, setting the RXNE flag and triggering an interrupt. The SPI ISR reads the received data from `SPIx_DR` and can load a new byte to transmit.

If even faster transmission is needed, the communication can be accelerated by using the transmit buffer empty flag (TXE) to trigger an interrupt. TXE is set when the transmit buffer is empty and ready to accept new data. This happens as soon as the transmit buffer is copied into the SPI shift register. The ISR needs to check the flags to determine which flag is set. If TXE is set, the ISR can load the new data into the transmit buffer immediately. The shift register is starting to receive the new data on the MISO signal, so that data is not available yet. However, if RXNE is set, then the data received from the previous transmission can be read from `SPIx_DR`.

Other Features

There are several other features available that we do not cover here. For example, the peripheral can transmit a CRC value as the last byte and automatically check the CRC value on the last received byte. This is useful for reliable communication.

Example: SPI Loopback Test

We can examine the basic SPI communication with a loopback test. We will connect the output data signal MOSI to the input data signal MISO. The program will then transfer a data byte. If the

³ But not all cases: the MCU’s interrupt response overhead is at least 16 cycles to enter the handler (register stacking, vector fetch) and at least five cycles to exit the handler (register unstacking). The SPI interface can run at up to one bit per two clock cycles: a byte every 16 clock cycles.

received data matches the transmitted data, the program will light the green LED. Otherwise the red LED will be lit.

The initialization code in Listing 8.5 configures the SPI1 peripheral and connects its signals to MCU pins. Figure 8.8 shows connection details: SCK is on PB3 (marked PWM/D3), MISO is on PB4 (marked D4), MOSI is on PB5 (marked PWM/D5) and NSS is on PA15. The baud rate is set to $48 \text{ MHz} / (22) = 48 \text{ MHz} / 16 = 3 \text{ MHz}$, or 333 ns per bit. Interrupts are not used. Be sure to connect MOSI (PB5) and MISO (PB4) for this test.

```
void Init_SPI1(void) {
    // Clock gating for SPI1 and GPIO A and B
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN | RCC_AHBENR_GPIOBEN;
    // GPIO A pin 15 in alternate function 0 (SPI1) for NSS
    // Set mode field to 2 for alternate function
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER15, ESF_GPIO_MODER_ALT_FUNC);
    // Select SPI1 (AF = 0) for alternate function
    MODIFY_FIELD(GPIOA->AFR[1], GPIO_AFRH_AFSEL15, 0);
    // GPIO B pin 3, 4, 5 in alternate function 0 (SPI1) for SCK, MISO, MOSI
    // Set each mode field to 2 for alternate function
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER3, 2);
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER4, 2);
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER5, 2);
    // Select SPI1 (AF = 0) for alternate function
    MODIFY_FIELD(GPIOB->AFR[0], GPIO_AFRH_AFSEL3, 0);
    MODIFY_FIELD(GPIOB->AFR[0], GPIO_AFRH_AFSEL4, 0);
    MODIFY_FIELD(GPIOB->AFR[0], GPIO_AFRH_AFSEL5, 0);

    // Clock is divided by 16 (2^(BR+1))
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_BR, 3);
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_MSTR, 1); // Master mode
    // Select first edge sample, active high clock
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_CPHA, 0);
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_CPOL, 1);
    // Data is LSB first
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_LSBFIRST, 1);
    // Data is 8 bits long
    MODIFY_FIELD(SPI1->CR2, SPI_CR2_DS, 7);
    // RXNE when at least 1 byte in RX FIFO
    MODIFY_FIELD(SPI1->CR2, SPI_CR2_FRXTH, 1);
    // Have NSS pin asserted automatically
    MODIFY_FIELD(SPI1->CR2, SPI_CR2_NSSP, 1);
    // Enable SPI
    MODIFY_FIELD(SPI1->CR1, SPI_CR1_SPE, 1);
}
```

Listing 8.5 Code to initialize SPI1 peripheral.

The function `Test_SPI_Loopback` is shown in Listing 8.6. It first calls `SPI_Send_Receive_Byte` (Listing 8.4) to exchange a byte of data with the slave using polled SPI communications. The LED is lit green if the sent and received data bytes match, otherwise it is lit red.


```
void Test_SPI_Loopback(void) {
    uint8_t out = 'A';
    uint8_t in;
    while (1) {
        in = SPI_Send_Receive_Byte(out);
        if (in != out) // Red: error, data does not match
            Control_RGB_LEDS(1, 0, 0);
        else // Green: data matches
            Control_RGB_LEDS(0, 1, 0);
        out++;
        if (out > 'z')
            out = 'A';
    }
}
```

Listing 8.6 Code to test SPI transmission and reception

The `main` function in Listing 8.7 initializes SPI1 and the GPIO ports for the LEDs, after which it calls the loopback test code.

```
int main(void) {
    Set_Clocks_To_48MHz();
    Init_GPIO_RGB();
    Init_SPI1();
    Test_SPI_Loopback();
}
```

Listing 8.7 Code to initialize MCU and run SPI loopback test.

Asynchronous Serial Communication

Protocol Concepts

Asynchronous serial communication works without a shared clock signal. Instead, both the transmitter and receiver have clock generators that must be configured to run at the same speed. The generic name for a peripheral that supports this is *universal asynchronous receiver/transmitter (UART)*. Such a peripheral often includes support for both asynchronous (no shared clock) and synchronous (with a shared clock) communication, so it may be called a *universal synchronous/asynchronous receiver/transmitter (USART)*.

universal asynchronous receiver/transmitter (UART)

Peripheral for asynchronous serial communications

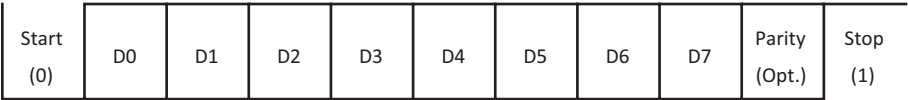


Figure 8.15 Contents of asynchronous serial communication message. The parity bit is optional, and there may be more than one stop bit.